PBEOT504: DATABASE MANAGEMENT SYSTEM

Last-Minute High-Yield Exam Booster & Query Cheatsheet

Master Summary of Schemas, ER Synthesis, Indexing Formulas, Relational Algebra & SQL Queries · Modules 1 & 2

1 MODULE 1: DBMS Architecture, ER Conceptual Modeling & Physical Storage

1.1 1. Three-Schema Architecture & Data Independence

- **Three Levels: External Level** (Tailored user views) → **Conceptual Level** (Community logical schema: entities, relationships, constraints) → **Internal Level** (Physical storage formats, block layout, access paths/indexes).
- **Logical Data Independence:** Ability to alter the **conceptual schema** (e.g., adding/deleting attributes or entities) without rewriting external schemas or application programs. *(Harder to achieve).*
- **Physical Data Independence:** Ability to alter the **internal schema** (e.g., building a B+ tree index, changing physical file organization) without changing the conceptual or external schemas. *(Easy to achieve).*

1.2 2. ER Modeling Elements & Notation Cheat Sheet

ER Element	Diagram Graphic	Exam Definition & Key Rules
Strong Entity	Single Rectangle	Possesses an independent primary key attribute (underlined).
Weak Entity	**Double Rectangle**	Has no key of its own; depends on owner entity via an **Identifying Relationship (Double Diamond)**. Has a **Partial Key / Discriminator (dashed underline)** and **always has Total Participation (Double Line)**.
Multivalued Attr	**Double Oval**	Attribute that can take multiple values for one entity (e.g., `Phone_Numbers`).
Derived Attribute	**Dashed Oval**	Computed from stored attributes (e.g., `Age` from `DOB`).
Participation	Total (Double Line) vs Partial (Single Line)	Total $\iff$ Every entity must participate ($min \geq 1$); Partial $\iff$ Optional ($min = 0$).
Ternary (Degree 3)	1 Diamond linking 3 Entities	Cannot be split into 3 binary relationships without losing atomic multi-way association.

1.3 3. Physical Storage & Indexing Formulas (Guaranteed Numerical Question)

Master Formulas for File Storage & Index Calculations

1. Blocking Factor $bfr = \lfloor B/R \rfloor$ [records/block]
2. Total Data Blocks $b = \lceil r/bfr \rceil$ [blocks]
3. Wasted Block Space $= B - (bfr \times R)$ [bytes]
4. Heap Linear Search Cost $= \lfloor b/2 \rfloor$ [average I/Os]
5. Primary Index (Sparse): One entry per data block (b entries); $R_i = V_K + P_B$; $bfr_i = \lfloor B/R_i \rfloor$; $b_i = \lceil b/bfr_i \rceil$; Binary Search Cost $= \lceil \log_2(b_i) \rceil + 1$.
1. Secondary Index (Dense on Key): One entry per data record (r entries); $b_i = \lceil r/bfr_i \rceil$; Search Cost $= \lceil \log_2(b_i) \rceil + 1$.
2. Multi-Level Index Tree: Fan-out $fo = bfr_i$;
   - Level 1: $b_1 = \lceil b/fo \rceil$
   - Level 2: $b_2 = \lceil b_1/fo \rceil \ldots$ until $b_x = 1$ (root)
3. Total Multi-Level Access Cost $= x + 1$ [block accesses].

2 MODULE 2: Relational Model, 7-Step Synthesis, Relational Algebra & Advanced SQL

2.1 1. Relational Keys & Integrity Constraints (Quick Definitions)

- **Super Key (SK):** Uniquely identifies a row (may have extra attributes). **Candidate Key (CK):** Minimal super key.
- **Primary Key (PK):** Candidate key chosen for identification. **NEVER NULL! Alternate Key (AK):** $CK \setminus \{PK\}$.
- **Foreign Key (FK):** Field matching PK of referenced table or NULL (Referential Integrity).
- **Entity Integrity:** $PK \neq$ NULL. **Domain Integrity:** Values must belong to atomic domain.

2.2 2. The 7-Step ER-to-Relational Synthesis Algorithm (Must Memorize!)

1. **Strong Entities:** Create table; flatten composite attributes into simple columns; $PK =$ Entity PK.
2. **Weak Entities:** Create table; import owner's PK as FK; composite $PK = \{\text{Owner } PK, \text{Partial Key}\}$.
3. **Multivalued Attributes:** Create separate table with $\{\text{Owner } PK_{\text{FK}}, \text{Attribute}\}$ as composite PK.
4. **Binary 1:1 Relationships:** Foreign Key Approach $\rightarrow$ Place PK of one side as FK on the side with **total participation**.
5. **Binary 1:N Relationships:** Place the PK of the 1-side as a **Foreign Key** inside the table on the N**-side**.
6. **Binary M:N Relationships:** Create a separate **Junction Table** with composite $PK = \{FK_1, FK_2\}$.
7. **Recursive 1:N / Ternary:** Add self-referencing FK in same table / Create 3-way junction table with composite PK.

2.3 3. Relational Algebra vs. SQL Master Query Cheatsheet

Operation	Relational Algebra (RA)	Equivalent SQL Query	Core Function
Selection	$\sigma_{\text{Salary}>50000}(\text{EMPLOYEE})$	`SELECT * FROM EMPLOYEE WHERE`	Filters rows conditionally
Projection	$\Pi_{\text{FName, LName}}(\text{EMPLOYEE})$	`SELECT DISTINCT FName, LName FROM EMPLOYEE`	Filters columns, eliminates duplicates
Theta / Inner Join	$\text{EMP} \bowtie_{\text{EMP.DNo = DEPT.DNo}} \text{DEPT}$	`SELECT * FROM EMP JOIN DEPT`	Combines matching rows
Natural Join	$\text{STUDENT} \bowtie \text{ENROLLS}$	`SELECT * FROM STUDENT NATURAL JOIN ENROLLS;`	Joins on common attribute names
Left Outer Join	$\text{EMP}]\bowtie \text{DEPT}$	`SELECT * FROM EMP LEFT JOIN`	Preserves unmatched left tuples (NULLs)
Division ($\div$)	$\Pi_{\text{RollNo, CNo}}(\text{ENR}) \div \Pi_{\text{CNo}}(\text{COURSE})$	`SELECT S.RollNo FROM STUDENT S WHERE NOT EXISTS (`	Universal quantification ("for ALL")

2.4 4. High-Yield SQL Templates for University Exams

Must-Know SQL Constructs

1. Aggregation & Grouping (WHERE vs HAVING):

```
SELECT DNo, COUNT(*) AS Total, AVG(Salary) AS AvgSal
FROM EMPLOYEE
WHERE Salary >= 20000.00
GROUP BY DNo
HAVING COUNT(*) >= 5 AND AVG(Salary) > 45000;
```

2. Nested Subqueries (IN vs EXISTS):

```
-- Non-correlated
SELECT FName FROM EMPLOYEE
WHERE DNo IN (SELECT DNumber FROM DEPARTMENT WHERE DName='EEE');

-- Correlated (Double NOT EXISTS for Division/ALL)
SELECT S.Name FROM STUDENT S WHERE NOT EXISTS (
    SELECT C.CourseID FROM COURSE C EXCEPT
    SELECT E.CourseID FROM ENROLLS E WHERE E.RollNo = S.RollN
```

3. View Creation:

```
CREATE VIEW HighEarners AS
SELECT SSN, FName, Salary FROM EMPLOYEE WHERE Salary > 60000;
```

4. Trigger Definition (ECA Rule):

```
CREATE TRIGGER validate_salary
BEFORE INSERT ON EMPLOYEE FOR EACH ROW
BEGIN
    IF NEW.Salary < 15000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary below minimum!';
    END IF;
END;
```

5. JDBC Architecture: `DriverManager` $\rightarrow$ `Connection` $\rightarrow$ `PreparedStatement` $\rightarrow$ `ResultSet`.